

VibeThinker evidence packet

Callosum interview reference | 3 October 2026

E0. Start here: claims, decisions, and where to look

This packet grounds the three-page report in **local VibeThinker-3B BF16 and NVFP4 results on an A100 PCIe 80GB**. Figures retain their recorded analyses. Measurements and hypotheses remain distinct; hosted Qwen and other models appear separately in Appendix A.

Interview question	Evidence to open	PDF page
What are the three latency targets?	E1: actual verification timeline	2
What did the experimental 30/60/120 tradeoff show?	E2: BF16 throughput tradeoff	3
Why buffer traces until the end?	E3: stacked CPU profile	4
What did dynamic fan-out and continuation achieve?	E4: allocation and verdicts	5
What supports NVFP4 and the backend choice?	E5: complete trial outcomes	6
Where is each report citation and its raw evidence?	E6: collated docket, original IDs 1-9	7
What happened with Qwen, CPU salvage, tools, and voting?	Appendix A: additional experiments	8
Did quantization improve decoding, TTFT, and first submission?	E7: run-level latency analysis	9

The strongest observed strategy comparison

The completed BF16 final-only pass@4 baseline reached 18 in **336.497s**. The initial BF16 coverage/continuation run reached 18 in **92.428s**: a **3.64x observed improvement**. Both used the same model, an 80% memory envelope, and 18 correct checks with zero wrong checks. Prompts, concurrency, seeds, budgets, context, and continuation differed; the comparison establishes the performance of complete strategies, rather than the isolated effect of any one change. [Baseline and paired strategy record](#).

Measurement contract

Time to 18 ends at the eighteenth distinct first positive verdict. Setup and warmup, cancellation settlement, and final buffered writes are separate. The accurate grader serializes all questions in a global FIFO queue, charging three seconds per check. Eighteen correct checks cost at least 54 seconds, before first-answer delay, wrong checks, and idle gaps.

TTFT is first output, not first answer. Tokens per second measure generation rate, CPU-seconds measure client work, and grader timestamps measure the verification path. These quantities cannot be added indiscriminately: inference, client work, and queue waits overlap.

Selected policy and status language. Frozen core v1 uses NVFP4/Marlin + FlashInfer, 30 x 1 barrier and four requests per question; dynamic60 is experimental. Use "observed" for individual runs and "not yet isolated" for causal effects. A stopped-at-18 run does not establish full accuracy or repeatability. No inference was launched for this report audit. [Current core and presets](#).

E1. Verification timeline: the three latency targets

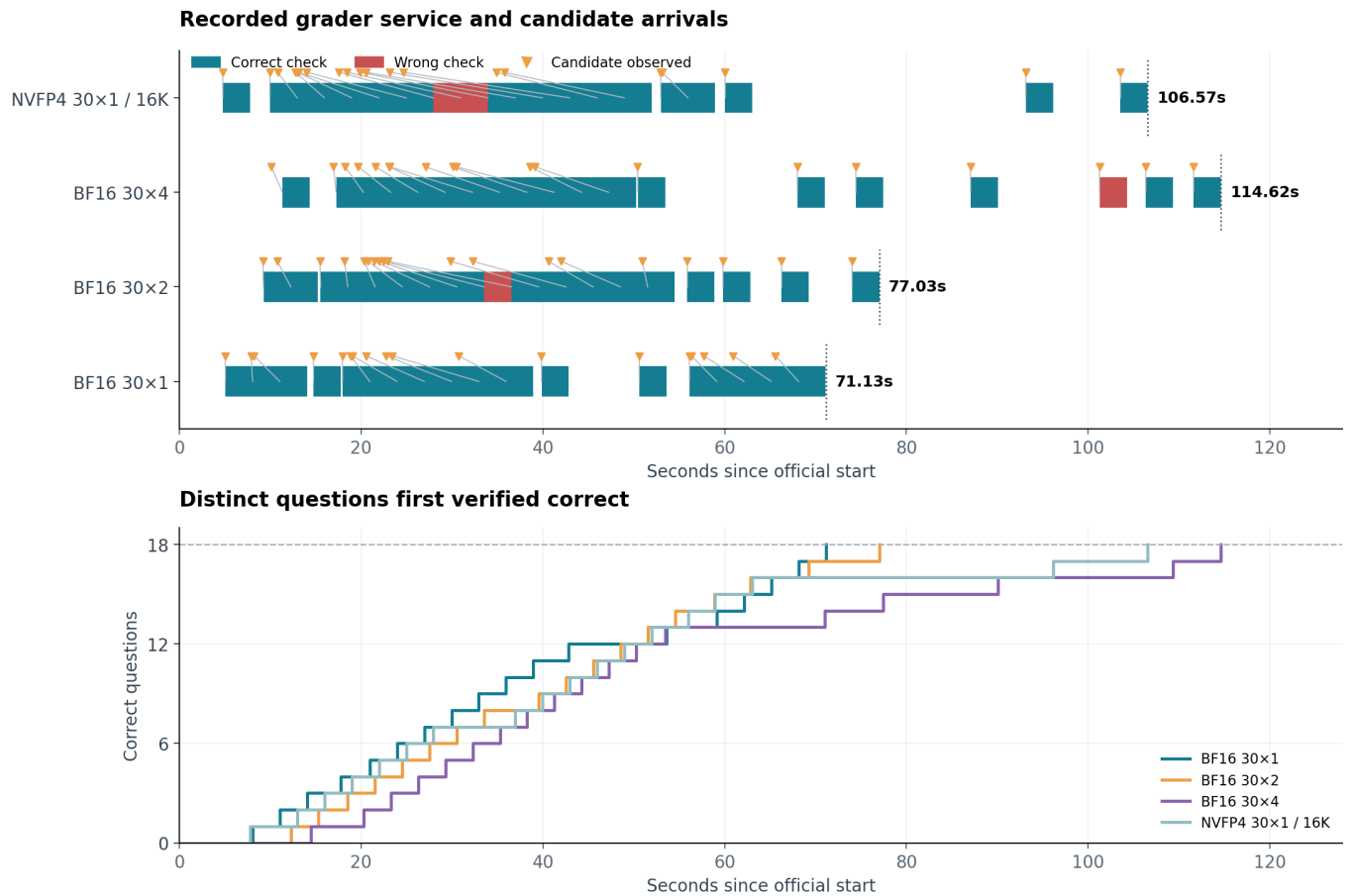


Figure 1. Recorded grader service, candidate arrivals, and distinct verified-correct questions

Read the plot. Orange triangles mark checked candidates; connectors show queue waits. Teal spans are correct checks, red spans wrong checks, and blank spans grader idle time. The lower panel counts distinct first successes.

Improvement target	Evidence in the plot	Engineering response
First usable answer	Time before the first triangle/check; TTFT alone misses this	Warm serving; start 30 x 1; expose intermediate proposals
Useful grader utilization	Blank spans and red checks between correct checks	Overlap decoding/grading; deduplicate; one pending check per question
Late-answer tail	Long gaps near the right edge; late steps toward 18	Recycle freed slots; try siblings and exact-ID continuations; cancel solved work

BF16 30 x 1: 5.055s to first pickup + 54.002s service + 12.076s idle gaps, approximately 71.135s. **BF16 30 x 4:** 11.332s + 57.002s + 46.279s, approximately 114.615s. The plotted NVFP4 16K trace is the **profiling-on 106.570s control**, with 60.002s service and two wrong checks, distinct from the 106.931s buffered rerun.

Decision. Bring useful candidates forward and reduce idle gaps. Each wrong check adds three seconds; aggressive filtering can delay correct candidates. Dynamic sampling targets the late gaps, but its effect needs measurement.

Source and audit: Runner profiling and benchmarking overhead; query-level times and CPU chart data. Original report references [3], [5]; docket E6.

E2. Why 60 active requests? An informed compromise

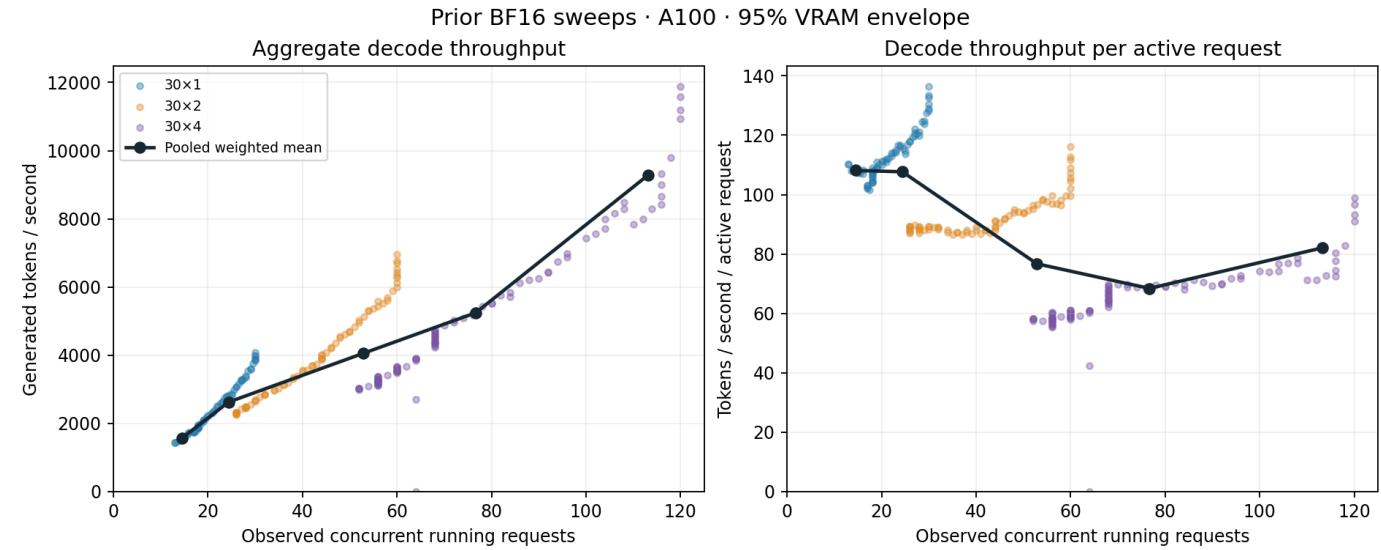


Figure 2. Prior BF16 sweeps: aggregate and per-active-request decode throughput

Observation. The left panel shows higher aggregate token output at wider concurrency. The right shows the cost to individual trajectories: the wider fan-out runs decode more slowly per active request. The pooled curve is not strictly decreasing; it rises again in the highest bin. The evidence supports a throughput/speed tradeoff, rather than a universal monotonic rule.

Initial sweep setting	Aggregate decode tok/s	Tok/s per active request
30 x 1	2,491	115.5
30 x 2	4,053	94.6
30 x 4	5,435	69.7

Experimental interpretation. Dynamic60 started one rollout per question for broad early coverage, then allowed up to 60 active requests as a middle ground between 30 and 120. Generation, source profiles and active question mix varied; 60 is not a measured optimum. The selected frozen core instead retains 30 x 1 barrier coverage and four requests per question. This experiment was motivated by BF16 observations, not fitted on the NVFP4 outcome.

Method and limits. Rates use engine-counter windows after initial first tokens, excluding warmup and windows with waiting requests at either endpoint. Windows crossing concurrency bins are excluded. Per-active rates use approximate request-seconds from sampled counts. Context lengths, question mix, cancellations, and trajectories vary; this is not a fixed-context scaling experiment. The 33-64 bin averaged 52.9 active requests, 4,056 aggregate tok/s, and 76.7 tok/s per request. It does not measure a constant 60-request workload.

The single dynamic60 NVFP4 run reached 18 at 127.565s, slower than the earlier 106.931s NVFP4 16K run. A matched repeated comparison of 30/60/90 slots would test the compromise; the current evidence does not establish 60 as best. Such a comparison would need consistent budgets, profiles, telemetry, and all outcomes retained.

Sources: Observed decoding throughput versus concurrency, window-level data, and BF16 sweep outcomes. Original report reference [3]; docket E6.

E3. Why buffer traces? Move avoidable CPU work

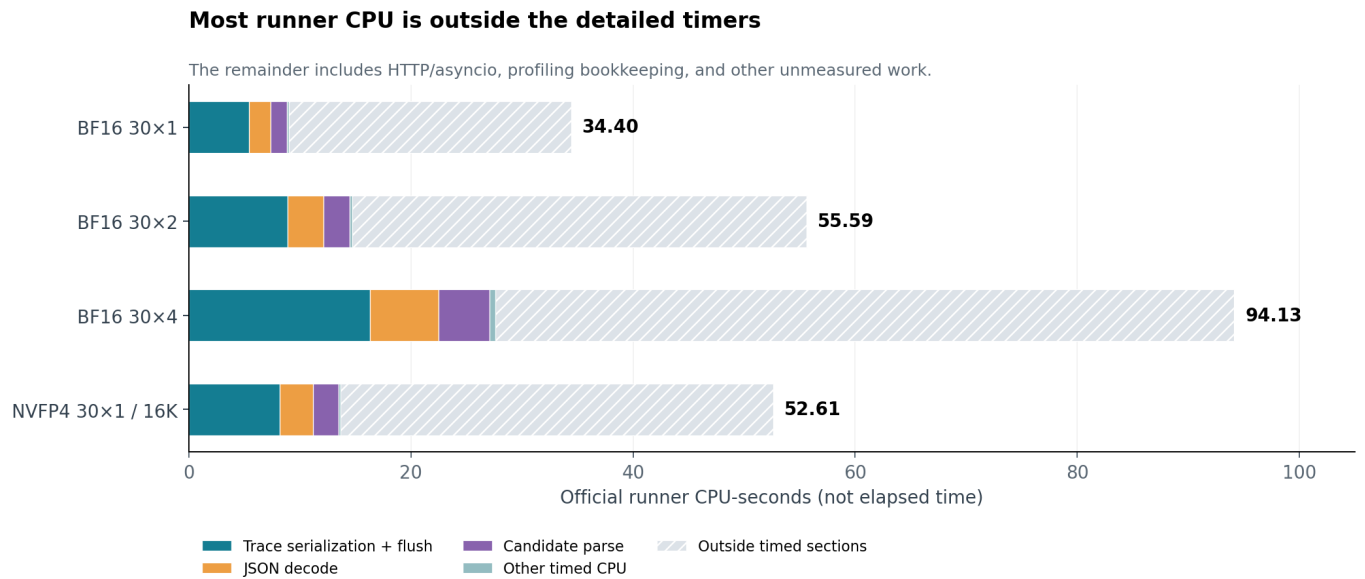


Figure 3. Stacked instrumented CPU times in the official solving window

Observation. Trace serialization/write/flush grows from **5.41 CPU-seconds at BF16 30 x 1** to **8.87 at 30 x 2** and **16.28 at 30 x 4**. Total client CPU grows from 34.40 to 55.59 to 94.13 CPU-seconds. JSON decoding and candidate parsing also grow. The hatched remainder includes HTTP/asyncio, stream handling, bookkeeping, and other unmeasured work; it cannot all be labeled profiling overhead.

Decision. Keep required stream rows, exact token IDs, candidates, and verdict timestamps in RAM while solving; serialize and flush them afterward. This removes synchronous trace persistence from the timed path. Live stream ingestion and candidate parsing still happen. Benchmark mode additionally disables optional CPU instrumentation, engine polling, and NVML sampling. Buffering alone and disabling profiling are separable controls; the combined trial does not isolate either one's effect.

Matched NVFP4 30 x 1 / 16K comparison	Profiling on	Buffered benchmark
Official runner CPU	52.606 CPU-s	37.509 CPU-s
First 18 verified correct	106.570s	106.931s
Final deferred flush	Immediate writes	1.943s after timing

The pair supports **28.7% lower measured CPU use**, without an observed end-to-end speedup. All initial request payloads and prompt IDs matched, but output trajectories changed. A local seven-repeat replay also reduced CPU by 27.7% with batched flushes and deferred meter aggregation; that replay excluded HTTP, GPU inference, and asynchronous concurrency.

Costs and measurement boundary. RAM buffering trades crash durability for less synchronous work. Graceful interruption flushes partial records; a hard crash can lose them. The measured objective excludes final flush, so retain its duration and report it if asked about total artifact-complete runtime. Optional GPU observations are unavailable in benchmark mode, rather than inferred from another run.

Sources: Runner profiling and benchmarking overhead, instrumented CPU data, and matched NVFP4 benchmark audit. Original report reference [5]; docket E6.

E4. Dynamic allocation and growing generation budgets

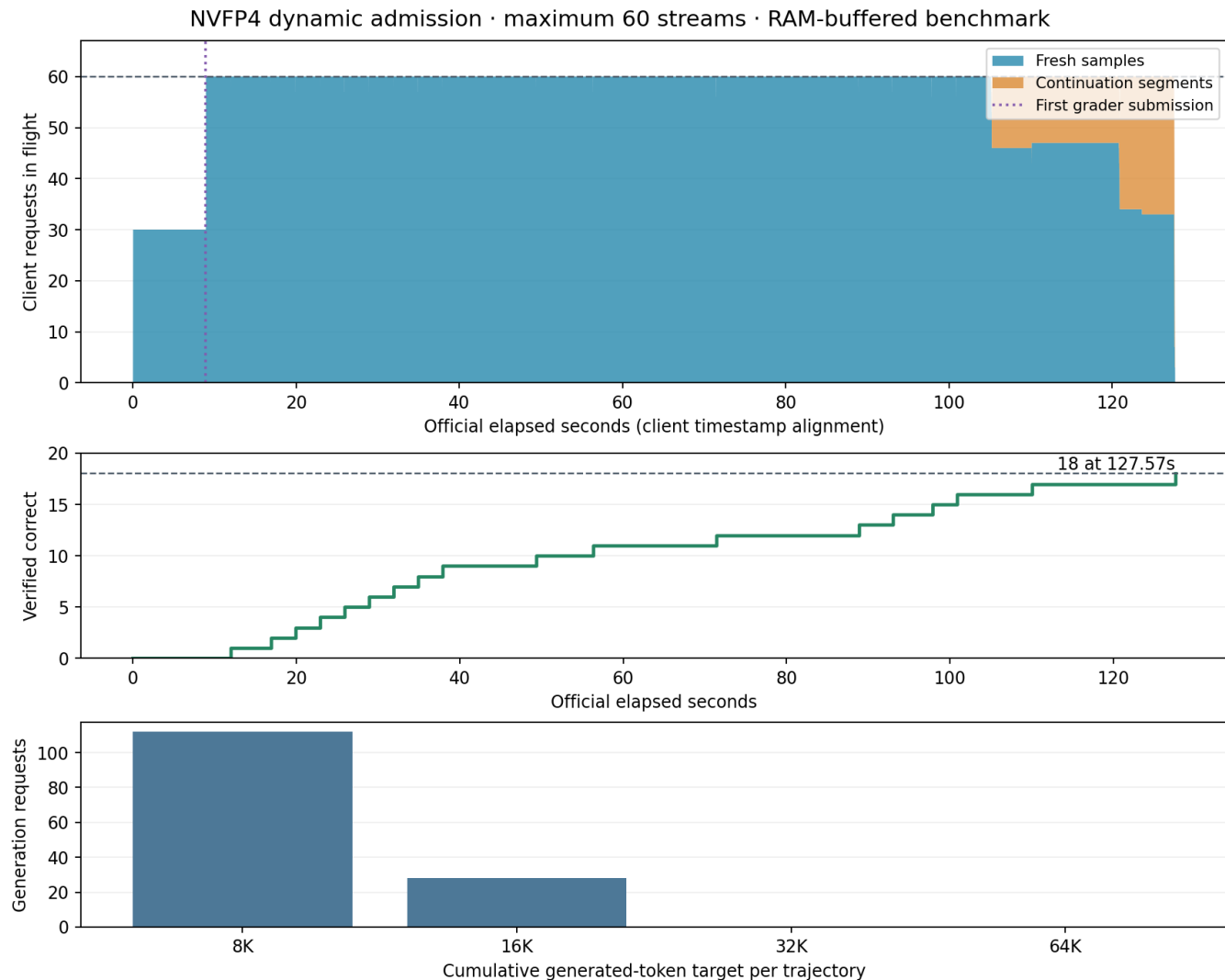


Figure 4. Dynamic60 admissions, concurrent streams, and first-solved verdicts

Implemented behavior. Thirty 8K streams start first. The first client grader submission, at **8.935s**, opens a pool of at most 60 active requests. Ready exact-ID continuations take priority; fresh 8K samples go to the least-active unsolved questions, with rotated ties. Solved questions release their slots and cancel siblings. One candidate queue and deduplication set span all samples for each question.

Observed outcome. The run used **112 fresh requests + 28 continuations**, reached 60 active streams, and gave some remaining questions up to five simultaneous samples. Every question stayed within eight requests, including continuations. Thirteen winners came from initial samples, four from extra fresh samples, and one from a continuation that supplied the eighteenth success. These identify winning requests, rather than counterfactual contributions.

The 8K/16K/32K/64K ladder is cumulative generated output, clipped to context including the prompt. All 28 actual extensions were **8K to 16K**; higher stages were unexercised. Prefixes matched exact IDs, but per-request cache-hit counts were unavailable and active KV retention was not established. The run reached 18 in **127.565s**, with 57.002s grader service and 61.573s idle gaps. Dynamic allocation worked; latency superiority remains unproven.

Audit: Run analysis, admission ledger, prefix/budget audit, and declared controls. Original report references [1], [6]; docket E6.

E5. Quantization, backend choice, and repeatability

Configuration. On this A100, the NVFP4 profile uses packed FP4 weights with Marlin and BF16 activations/KV. The later FlashInfer experiment changes the attention backend. These local results ground the choice; Blackwell native-FP4 throughput claims do not establish performance here. [Pinned model and serving profile](#).

Completed comparison	All first-18 times (s)	Median (s)
BF16 30 x 1, benchmark repeats	117.17 / 134.52 / 135.65	134.52
BF16, profiling restored	116.28 / 150.07 / 152.01	150.07
NVFP4 30 x 1, paired repeats	115.40 / 96.67 / 96.67	96.67
BF16, dynamic30 policy	92.01 / 105.40 / 114.30	105.40
BF16, exact original source	102.99	Single trial
NVFP4, four declared different seeds	118.19 / 94.59 / 126.43 / 114.44	116.31
NVFP4 + FlashInfer, development	98.327 / 64.453 / 64.399	64.453

Decision and strength of evidence. The paired NVFP4 batch had a lower median than BF16, making it worth pursuing. It did not reproduce 71.135s. Seventeen follow-ups before the backend change failed to recover that historical result. A different-seed batch also failed to do so. Matching seeds, payloads, and prompt IDs does not freeze realized reasoning paths.

The FlashInfer development batch produced two runs below 71.135s, while retaining the slower first run. The first trial's median initial TTFT was **3.510s**, versus **0.198/0.240s** on the later trials. A short synthetic warmup did not establish that every official prompt path was fully settled. Startup and warmup costs remain separate from official solving time.

Why the faster trials matter. They each needed one wrong check and 13 continuations, yet grader idle gaps fell to **2.611/2.473s**. The grader spent about 57 seconds on service, so the improvement is consistent with useful candidates arriving fast enough to keep it busy. This ties directly to E1. Attention math can alter sampled trajectories; these results do not isolate a pure decode-speed or quality improvement.

Completed validation. The settling trial took **136.048s**; the three scored trials took **87.356/59.316/86.574s** (median 86.574s). Only one beat 71.134766841s, so the predeclared all-three gate failed. Prefix caches were reset before warmup, with a fresh grader and every outcome retained. The fastest scored run had 54.003s service and 0.483s idle; the other two had 25.657/27.739s idle. [Validation records](#). E7 separates decoding, TTFT, and first submission.

Sources: [Full replication comparison](#), [paired NVFP4 trials](#), [declared-seed outcomes](#), [FlashInfer development outcomes](#), and [validation protocol](#). Original report reference [\[4\]](#); docket E6.

E6. Collated docket: preserve the report's citation IDs

Use these original IDs when a claim in the main report is challenged. Each entry gives the question it answers, packet location, and a route to the recorded evidence. Links point into the workspace; the PDF embeds all figures so the visual evidence travels with it. Copied-figure hashes and provenance.

[1] Frozen policy and historical v4 -> E0/E4, pages 1/5

Claim: the selected frozen core uses 30 x 1 barrier coverage, four requests per question, streamed extraction/deduplication, one pending grader request per question, exact-ID continuation and cancellation. Frozen core contract and presets. V4 admission rules below describe a separate experimental policy. Runner catalogue; canonical semantics and timing; v4 allocation implementation; v4 declared configuration.

[2] Intermediate-answer replay -> Appendix A, page 8

Claim: earlier answers motivated the live runner; the hosted Qwen pass@4 replay estimates 315.95s to 177.10s with 120 simultaneous trajectories. Replay report and limits; machine-readable replay. This is additional-model simulation evidence.

[3] Concurrency tradeoff -> E1-E2, pages 2-3

Claim: wider BF16 fan-out raises aggregate output and costs per-trajectory speed; 60 is a heuristic. Throughput figure/method; window data; sweep outcomes, KV, preemptions, CPU.

[4] Quantization and repeatability -> E5, page 6

Claim: NVFP4 decoding improved; TTFT did not, and first-submission significance remains unestablished. Complete comparisons; comparison data; development; validation. E7 adds run measurements, statistics/provenance, backend log excerpts/hashes, and reproduction script.

[5] Client CPU and grader activity -> E1/E3, pages 2/4

Claim: trace work grows with attempts; deferring writes reduces measured CPU, while the matched pair did not improve wall time. CPU profile, verification timeline, replay; query times and CPU totals; buffered-pair audit.

[6] Dynamic60 and continuation -> E4, page 5

Claim: 60 slots and exact-ID continuation worked, but only 8K-to-16K stages ran and the trial was slower. Run report; allocation ledger; first-solved events; prefix audit.

[7] Alternative extraction, stopping, and voting -> Appendix A, page 8

Claim: small-model semantic judgments and voting did not warrant adoption. CPU salvage; hosted extraction/verification; prompt pilot; marker audit; Jev calibration; self-consistency; Reflex-8.

[8] Python tools -> Appendix A, page 8

Claim: optional Python improved observed hosted Qwen coverage; local VibeThinker latency needs validation. Matched comparison and replay limits; paired replay summary.

[9] Completed baseline comparison -> E0, page 1

Claim: 336.497s versus 92.428s is a measured 3.64x complete-strategy comparison. Baseline with paired comparison; baseline summary; coverage records. The coverage README's 92.503s includes cancellation settlement; 92.428s is first-18 timing.

A1. Appendix: additional models and hosted experiments

These experiments informed the design but are not local VibeThinker latency measurements. Their results should be described as motivation, diagnostic evidence, or targets for local replication. Original citation IDs [2], [7], [8] remain traceable through E6.

Intermediate extraction and prompt-only stopping [2, 7]

The hosted Qwen3-30B-A3B first-four-sample replay covered 18/30 using final answers and 22/30 using permissive intermediate candidates. At 120 simultaneous saved trajectories, estimated time to 18 fell from 315.95s to 177.10s; a 30-slot replay estimated 835.19s to 235.48s. Delivery times were inferred from token fractions, and saved durations were reused across capacities. Neither is a live A100 throughput measurement. A streamed prompt pilot showed that asking for the first answer did not reliably stop rechecking, and conservative final-answer markers missed earlier prose candidates.

CPU salvage, semantic extraction, and model verification [7]

Local CPU Gemma added no quote-grounded candidates beyond syntax recovery in the selected controls; median tail extraction was roughly 9-15s and full-trace reads were much slower. Hosted Gemma/Qwen chunk classifiers measured lower service latency, but scope judgments depended on prompt/formatting guards and small reused controls. The hosted Qwen 2.5 7B verification probe approved five of six correct candidates and **five of seven wrong ones**, leaving it unsuitable as a correctness certificate. These findings motivate deterministic extraction plus the accurate provided grader.

Jev trajectory scoring [7]

Low rejection thresholds discarded little work; stronger thresholds missed correct completions. A 70% threshold rejected two of 54 observed successes within the specified continuation budget, while a 60% threshold rejected 16 eventually correct completions under the longer original cap. Capped traces were censored rather than proven unsalvageable. No live speedup from confidence-based pruning was established.

Optional Python with hosted Qwen3.5 [8]

Final-only coverage was 19/30 with optional Python versus 15/30 without; matched permissive extraction yielded 24/30 versus 22/30. A 30-slot replay estimated 1.77 versus 2.93 minutes to 18. Original runs used eight active trajectories and occurred at different times; the replay does not establish live 30-slot latency. VibeThinker tool-use reliability and arithmetic/code quality require independent evaluation before adoption.

Self-consistency and Qwen3.5-4B Reflex-8 [7]

In the Qwen3 eight-sample cohort, pass@8 covered 22/30, a unique modal vote 21/30, and a strict majority 15/30. Waiting for votes adds latency. The no-thinking Qwen3.5-4B probe produced only two questions with four-of-eight agreement, neither correct, and no correct unique top-vote answer. Accurate cheap grading favors independent candidate checking over waiting for consensus in this task.

What remains to transfer locally

Replicate the answer-arrival and Python findings on the scored GPU/model before treating them as VibeThinker gains. Different-model capability comparisons should hold grader policy, question set, timing boundaries, and candidate extraction constant, while recording model/provider/context differences. Prepared Qwen3.5 A3B GPTQ coverage plan is a preparation record, not a completed result in this packet.

Direct source routes: replay, prompt pilot, CPU salvage, semantic sidecars, Jev, Python comparison, voting, and Reflex-8.

E7. Quantization: three separate claims

Recorded VibeThinker 30 x 1 / 8K batches: first answer versus later grader starvation

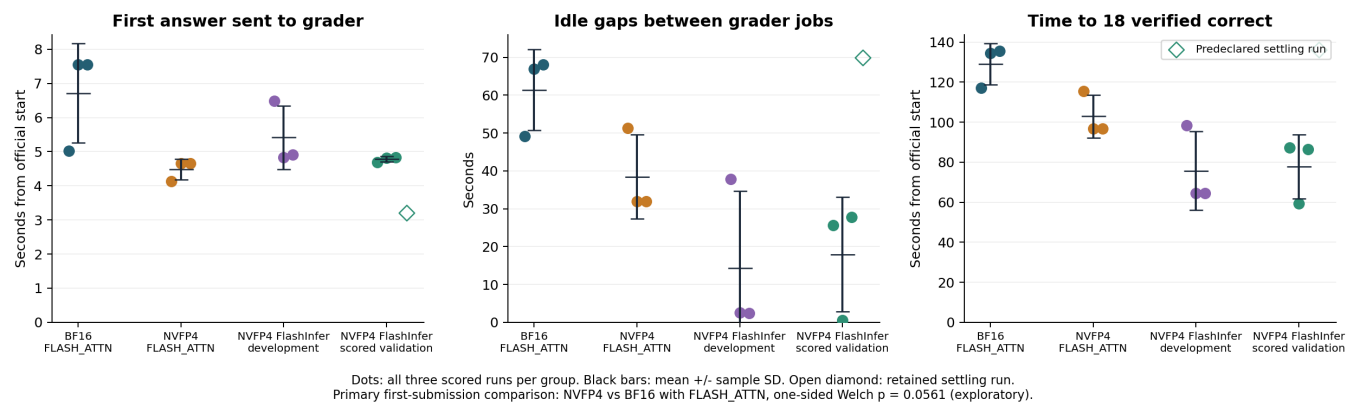


Figure 5. First submission, subsequent grader idle, and time to 18 across all recorded batches

Primary comparison: three 30 x 1 / 8K warmed runs per profile, with FLASH_ATTN fixed. Saved server logs confirm FLASH_ATTN for BF16 and standard NVFP4, including the earlier BF16 best run. FlashInfer was an explicit later change. [Backend excerpts and hashes.](#)

Metric	BF16	NVFP4	Supported claim
Observed decode rate	112.5 tok/s	136.6 tok/s	+21% received-token rate; workload observation
Initial-request TTFT	163ms	205ms	26% slower; no TTFT improvement
First grader submission	6.713s	4.482s	33% lower mean; one-sided Welch p=0.056

Definitions. Decode and TTFT entries average each run's median over 30 initial streams. Approximate decode rate is received token IDs divided by the interval between first and last output delta; it includes the first delta's tokens. Cancellation, changing active load, contexts, and trajectories affect this rate. It is not fixed-concurrency GPU throughput. First submission is the earliest client verification-start timestamp minus official run start, before grader queuing or service.

Statistical boundary. BF16 first-submission times were 5.028/7.555/7.557s; NVFP4 times were 4.125/4.662/4.660s. One-sided unequal-variance Welch gives $t=-2.590$, $df=2.179$, $p=0.0561$. The NVFP4-minus-BF16 mean difference is -2.231s; its one-sided 95% upper bound is +0.149s. This does not establish improvement at $\alpha=0.05$. The analysis is retrospective, with only three runs per group, sequential model batches, shared warmed servers/fixed seeds, and different source commits. Trial numbers do not justify pairing; 30 concurrent questions are not 30 independent runs.

FlashInfer interpretation. Scored validation averaged 146.7 tok/s, 189ms TTFT, and 4.780s first submission. Its strongest first-18 result came with only 0.483s later grader idle. Faster decoding and useful later arrivals remain promising; a first-submission improvement over standard NVFP4 is not observed here, and the repeatability gate failed.

Next test: randomized interleaved blocks across model/backend combinations, predeclared seeds and endpoints, consistent warmup, and repeated server starts. Analyze run-level outcomes; use paired differences only for deliberately matched blocks.

Audit: per-run measurements, initial-stream counts/rates, tests, caveats, and input hashes, reproduction script. Method: SciPy Welch/one-sided test. Original citation [4]; docket E6.